Supervyse OPF AST26X0 Quick Start Guide

Revision 1.3

03/2025

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde

Revision History 4
Introduction 5
Download SDK from bitbucket 6
Setup App passwords for bitbucket on development machine 6
Install google repo tool on development machine 6
1.1.1 Download from Google 7
1.1.2 Download from 清华大学: 7
Setup git user information on development machine 7
Download OPF ast26x0 SDK 6
Setup of the Build Environment 8
Install the packages on your Ubuntu 18.04/64 & 20.04/64 8
Install the packages on your Ubuntu 22.04/64 8
Link /bin/sh to bash 8
Build Source Code 10
ast26x0evb 10
1.1.3 ast2600evb 10
1.1.4 ast2620evb 10
Build BMC firmware for QEMU 10
Signing BMC Firmware 12
With Default Private Key 12
With Specific Private Key 12
Build performance 14
Troubleshooting 15

Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2022-10-12	-	-	RV22.11 Release Document
1.1	2023-06-20	Ben Chang	Conrad Yao	Add Ubuntu 22.04 support
1.2	2024-05-14	Youmin Chen	Edwin Liu	Add "Build BMC firmware for QEMU" section
1.3	2025-03-14	Ben Chang	Conrad Yao	Added RV24.12 manifest file description

Introduction

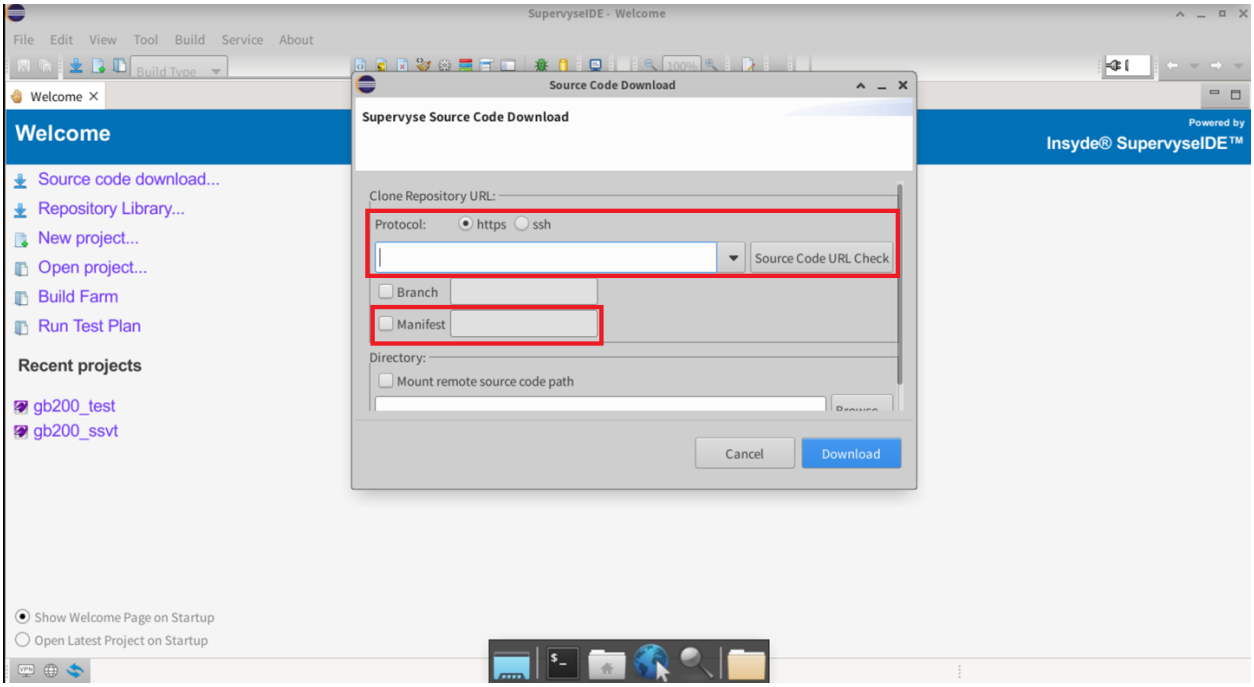
This page introduces how to setup a build environment for compiling Supervyse OPF code, including the software packages you need to install.

Download SDK from IDE

Supervyse OPF is a Larger-scale project with many git repositories. Insyde uses Google repo tool to manage Supervyse OPF. Users can directly use IDE to get the required code.

Use IDE to Download SDK Guide

Please refer to the **Download Source Code** section in the **SupervyseIDE_OPF_User_Guide.pdf** file included in the SupervyseIDE installation package.



Download OPF ast26x0 SDK

In the IDE Download Source Code window, you can select the download url:

```
https://bitbucket.org/insydesw/g4_manifest-ast2600evb
```

Manifest xml file	Description
AST2600_OPF-ast2600evb_master.xml	The manifest xml file is used for download of development branch. It will be updated to the latest status and the stability cannot be guaranteed.
AST2600_OPF-ast2600evb_RV24.12.4.xml	The manifest xml file is used for downloading of tag RV24.12.4.xml

Refer to the table above to select the manifest you need:

AST2600_OPF-ast2600evb_master.xml

AST2600_OPF-ast2600evb_RV24.12.4.xml

Setup of the Build Environment

Before using the SDK, you should consider what your needs/requirements will be.
Supported Operating System list:

- Ubuntu 18.04/64 (EOS in June 2023)
- Ubuntu 20.04/64
- Ubuntu 22.04/64

Install the packages on your Ubuntu 18.04/64 & 20.04/64

Please install the following tools.

```
$ sudo apt-get update
$ sudo apt-get install -y git build-essential libsdl1.2-dev texinfo gawk chrpath diffstat zstd pigz
$ sudo apt-get install -y unzip gcc-multilib g++-multilib xxd curl liblz4-tool
$ sudo apt-get install -y python jq python3-pip
$ sudo -H pip3 install yq
```

Install the packages on your Ubuntu 22.04/64

Please install the following tools.

```
$ sudo apt-get update
$ sudo apt-get install -y git build-essential libsdl1.2-dev texinfo gawk chrpath diffstat zstd pigz
$ sudo apt-get install -y unzip gcc-multilib g++-multilib xxd curl liblz4-tool
$ sudo apt-get install -y jq python3-pip
$ sudo -H pip3 install yq
```

Link /bin/sh to bash

You must use bash instead of sh to build the code

```
$ sudo dpkg-reconfigure -plow dash
```

Choose '**No**' in the 'Configuring dash' popup dialog and press enter.

If it doesn't work, try:

```
$ sudo rm /bin/sh

$ sudo ln -s /bin/bash /bin/sh
```

- To ensure that the link was changed to bash:

```
$ ls -l /bin/sh
```

Build Source Code

ast26x0evb

1.1.1 ast2600evb

```
$ source setup evb-ast2600-insyde
$ bitbake obmc-phosphor-image
```

The output image with time stamp, "obmc-phosphor-image--xxxxxxxxxxxxxx.static.mtd, will be placed in:

/build/evb-ast2600-insyde/tmp/deploy/images/evb-ast2600-insyde

To find the latest image quickly, the latest output image is linked to the following synbolic link:

/build/evb-ast2600-insyde/tmp/deploy/images/evb-ast2600-insyde/image-bmc

1.1.2 ast2620evb

```
$ source setup evb-ast2620-insyde
$ bitbake obmc-phosphor-image
```

The output image with time stamp, "obmc-phosphor-image--xxxxxxxxxxxxxx.static.mtd, will be placed in:

/build/ evb-ast2620-insyde/tmp/deploy/images/evb-ast2620-insyde

To find the latest image quickly, the latest output image is linked to the following synbolic link:

/build/evb-ast2620-insyde/tmp/deploy/images/evb-ast2620-insyde/image-bmc

Build BMC firmware for QEMU

- Since QEMU cannot simulate AST2600 perfectly, an user have to disable some unsupported hardware features to prevent QEMU from terminated abnormally.
- If an user would like to test firmware on QEMU environment, enable the flag IS_BUILDING_QEMU_IMAGE at local.conf file

- Take ast2600 evb as an example, after setup machine, enable the flag in /build/intel-ast2600-evb/conf/local.conf

```
# Enable this flag if an user wants to test firmware image on QEMU.
# With this flag some unsupported hardware feature will be disabled
# to prevent QEMU hang
IS_BUILDING_QEMU_IMAGE = "1"
```

Signing BMC Firmware

With Default Private Key

- OPF will use the following default private key to sign BMC firmware.
- The private key is used to sign the image.

<SDK>/meta-phosphor/recipes-phosphor/flash/files/OpenBMC.priv

- The publickey corresponding to private key is in the following location.
- The publickey is used to verify whether the image is valid or not when firmware update is procceeding.

<SDK>/build/tmp/work/$MACHINE-openbmc-linux-gnueabi/$RECIPE/$VERSION/rootfs/etc/activationdata/key/publickey

EX: (AST26x0)

<SDK>/build/tmp/work/evb_ast26x0_insyde-openbmc-linux-gnueabi/obmc-phosphor-image/1.0-r0/rootfs/etc/activationdata/OpenBMC/publickey

- If you decompress the BMC firmware image, there are some .sig files which are used to be verified with publickey

    1. image-full.sig
    2. image-kernel
    3. image-kernel.sig
    4. image-rofs
    5. image-rofs.sig
    6. image-runtime
    7. image-runtime.sig
    8. image-rwfs
    9. image-rwfs.sig
    10. image-u-boot
    11. image-u-boot.sig
    12. MANIFEST
    13. MANIFEST.sig

With Specific Private Key

- If you want to use other private key, not default one. You can use following way to produce your own private key.

:* Generate RSA1024 private key named "OpenBMC.priv"

```
# openssl genrsa -out OpenBMC.priv 1024
```

:* Generate RSA2048 private key named "OpenBMC.priv"

```
# openssl genrsa -out OpenBMC.priv 2048
```

:* Generate RSA4096 private key named "OpenBMC.priv"

```
# openssl genrsa -out OpenBMC.priv 4096
```

- Export some environment variables
- Build firmware image with environment variables

```
# export BB_ENV_EXTRAWHITE="$BB_ENV_EXTRAWHITE SIGNING_KEY"
# SIGNING_KEY=/path/to/secure/OpenBMC.priv bitbake obmc-phosphor-image (or intel-platforms)
```

- **It's a normal behavior when BMC Firmware update failed after private key has been changed.**

Build performance

- By default, Yocto use all CPU of the build machine to do the jobs.

If user want to specify the number of the jobs it can perform,

add following options to /conf/local.conf

```
BB_NUMBER_THREADS = "4"
PARALLEL_MAKE = "-j 4"
```

- Yocto uses build/downloads and build/sstate-cache as your default directories.

You can change them by adding the following options to build/conf/local.conf.

This will increase speed when you rebuild the same project with the same settings.

```
BB_GENERATE_MIRROR_TARBALLS = "1"
DL_DIR="/where/to/your/downloads"
SSTATE_DIR="/where/to/your/sstate-caches"
```

Troubleshooting

It is suggested that you double-check your versions of both xq and yq before you start to build OPF.

On Ubuntu 18.04:

```
$ xq --version
xq 2.12.0
$ yq --version
yq 2.12.0
```

On Ubuntu 20.04:

```
$ xq --version
xq 3.2.1
$ yq --version
yq 3.2.1
```

On Ubuntu 22.04:

```
$ xq --version
xq 3.2.2
$ yq --version
yq 3.2.2
```

Try upgrading your xq or yq if their versions are much lower than above. You may need to uninstall your local versions first.

```
$ pip3 uninstall xq
$ pip3 uninstall yq
$ sudo -H pip3 install yq
```

Then, exit and login again to take effect.